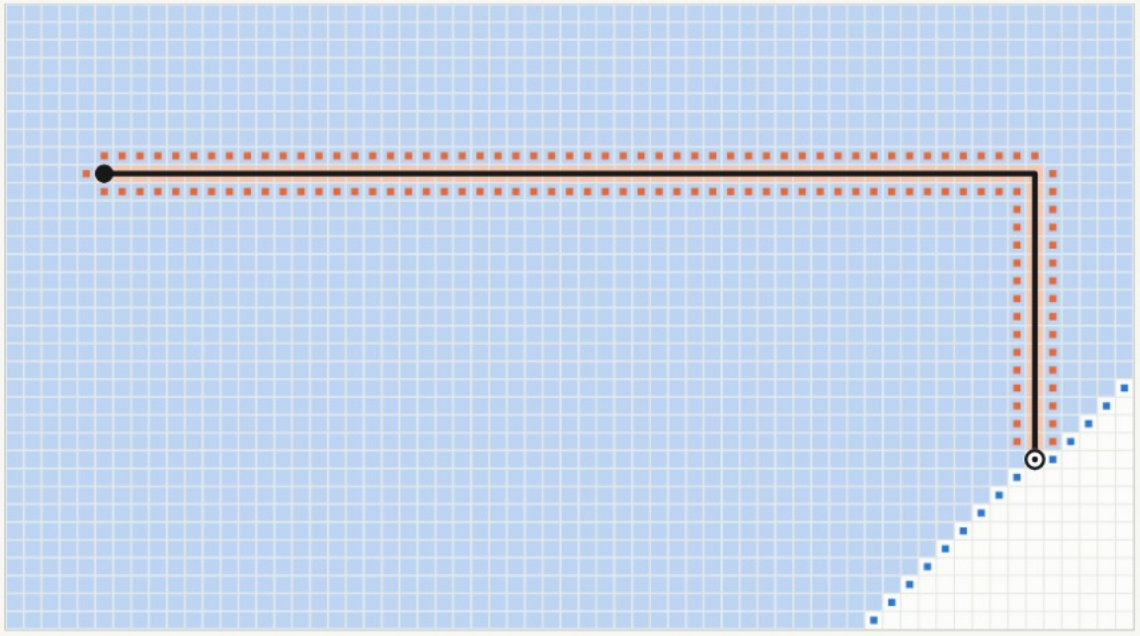
ADSA · PROJECT T01 · UFAZ · FALL 2026

When does A* save work?

Dijkstra and A* on grids with obstacles and on the California road network. They return the same shortest paths, but do very different amounts of work to find them.

Elnur Haqqverdiyev · Rajab Iskandarli

M1 Data Science and Artificial Intelligence



THE QUESTION

Both algorithms return the same shortest path. The difference is how much of the graph they look at.

MEASURE 1 · WORK

Expanded vertices: how many vertices the search settles before it reaches the target.

MEASURE 2 · COST

Wall-clock time: nanoseconds around one search call, median of repeated runs.

COMPARISON

Always **paired**: both algorithms run on the same graph, the same source and target. Ratio = Dijkstra ÷ A*, so above 1 means A* wins.

$$\text{time} = \text{expansions} \times \text{cost per expansion}$$

So A* is faster only when

$$\frac{\text{expansions(Dijkstra)}}{\text{expansions(A*)}} > \frac{\text{cost(A*)}}{\text{cost(Dijkstra)}}$$

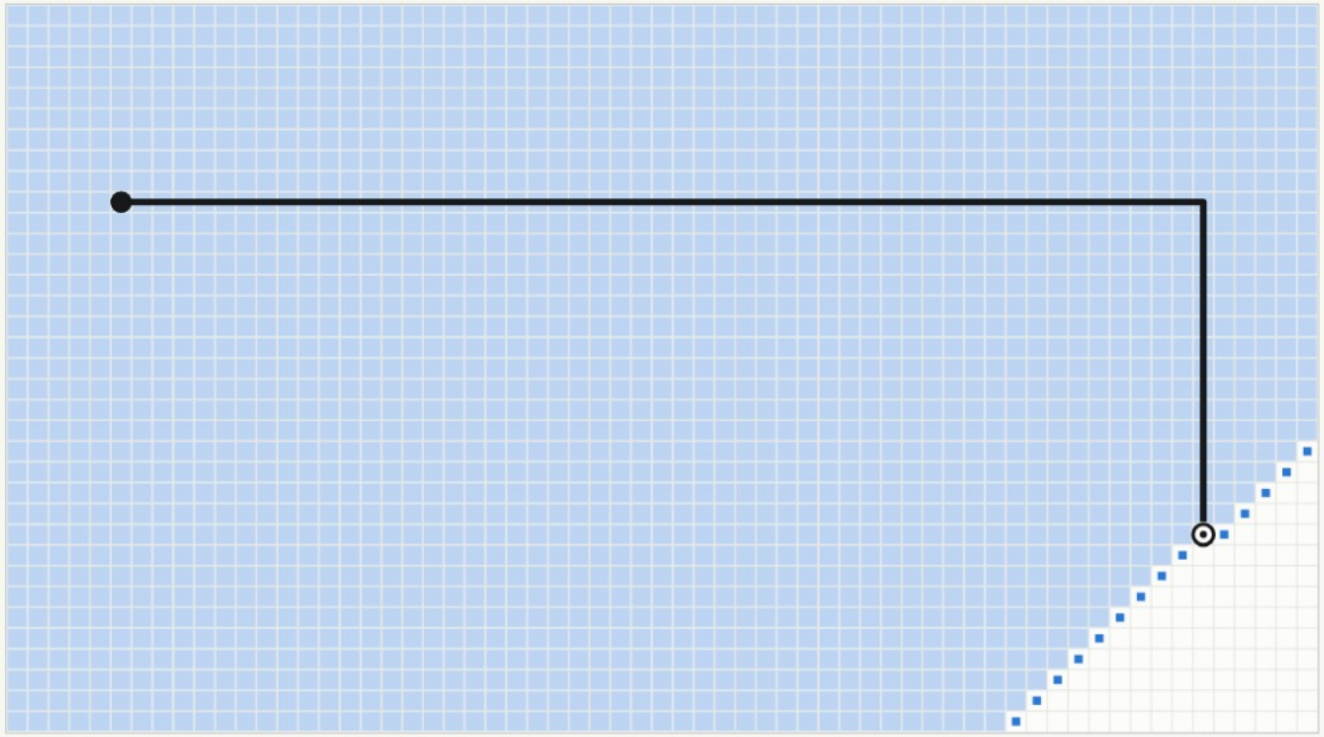
One A* expansion costs **1.2 to 2.0×** a Dijkstra expansion in our runs: for every vertex it settles, A* pushes more entries into the heap and computes h for each of them. The rest of the talk is about when the left side is bigger.

illustration · 63 × 35 grid

Dijkstra settles vertices in order of distance from the start

Dijkstra `f = g`

SETTLED **2,091**



REPLAY PLAY

Settled **2,091** cells to find a **68**-step path.

Priority **`f(v) = g(v)`**, the distance walked so far. The closest unsettled vertex is always next.

The frontier grows like a ripple, the same in every direction. Dijkstra has no idea where the target is.

```
pop u with the smallest f
if u is already settled: skip it
if u is the target: stop
for each neighbour v of u:
  if g(u) + w(u,v) < g(v):
    g(v) = g(u) + w(u,v)
    push v with priority f(v)
```

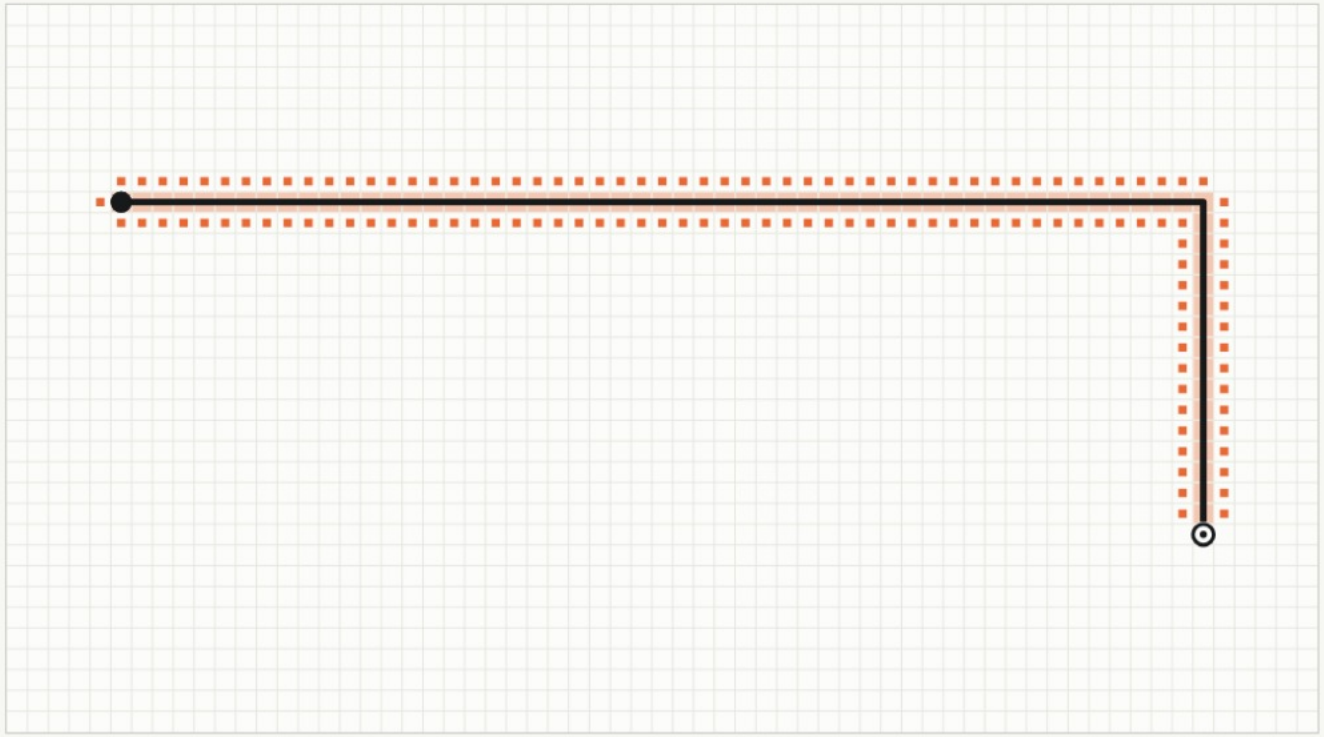
settled · waiting in the heap · shortest path

illustration · same grid, same start and target

A* adds a guess of the remaining distance

A* `f = g + h`

SETTLED **69**



REPLAY PLAY

Settled **69** cells to find a **68**-step path.

Priority **`f(v) = g(v) + h(v)`**: distance walked plus a guess of what is left, so f estimates the whole path through v. On a grid the guess is **Manhattan distance**: steps left or right plus steps up or down.

8	7	6	5	4	3	2	1	2
7	6	5	4	3	2	1	◎	1
8	7	6	5	4	3	2	1	2
9	**8**	7	6	5	4	3	2	3
10	9	8	7	6	5	4	3	4

h to the target ◎ · outlined cell: h = 8, and with no walls the true distance is also 8

It never overestimates, so A* still returns the **exact** shortest path. It changes by at most 1 per step, so no vertex is settled twice.

Ties in f are broken toward the larger g (`larger_g`): keep walking the path you are on.

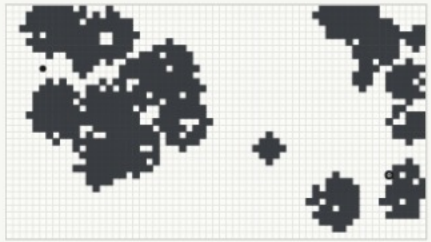
Five grid structures and one real road network



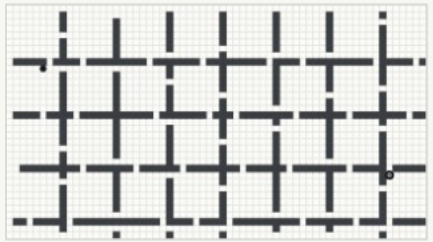
open · no walls



random · 20% walls



clustered · 30% in blobs



rooms · walls with doors



maze · one route only

GRIDS

Each family at 128², 256², 512² and 1024² cells, 4-neighbour, unit cost.

ROAD NETWORK

SNAP roadNet-CA: 1,957,027 intersections, 2,760,388 road segments, unit hop cost.

ONE SEARCH ROUTINE

Dijkstra is the A* code with h = 0: same heap, same loop, same counters.

QUERIES

15 per grid, 30 on roads, in short, medium and long distance bands chosen by BFS.

TREATMENTS

Grids: Dijkstra, A* (larger_g), A* (node_id). Roads: Dijkstra, ALT with K = 1, 4, 8, 16. 3,600 timed runs.

CORRECTNESS

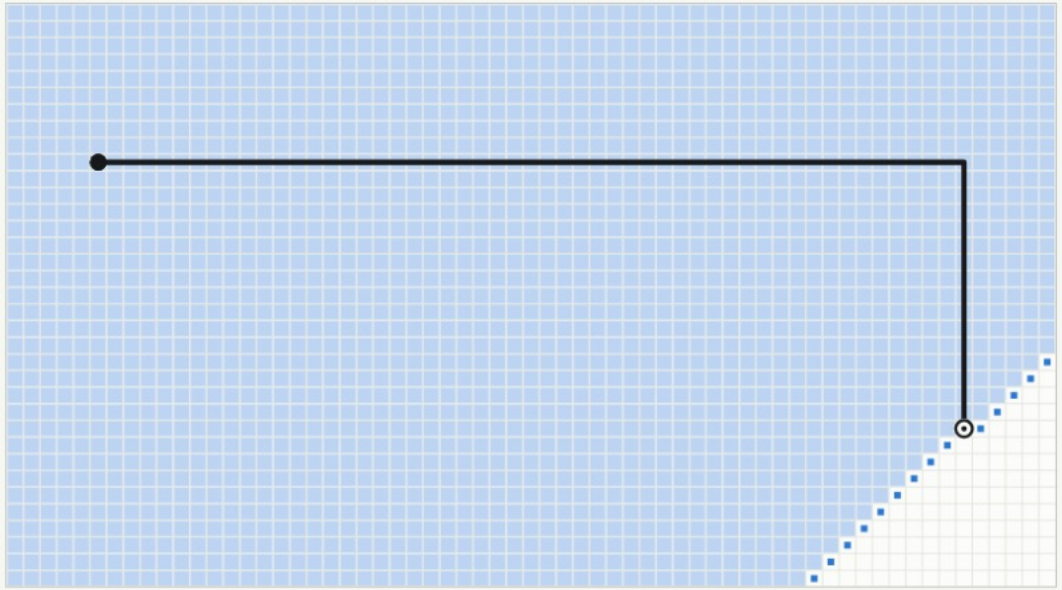
3,600 of 3,600 runs matched an independent BFS distance. No timeouts.

numbers: 60 queries, 128² to 1024², medians

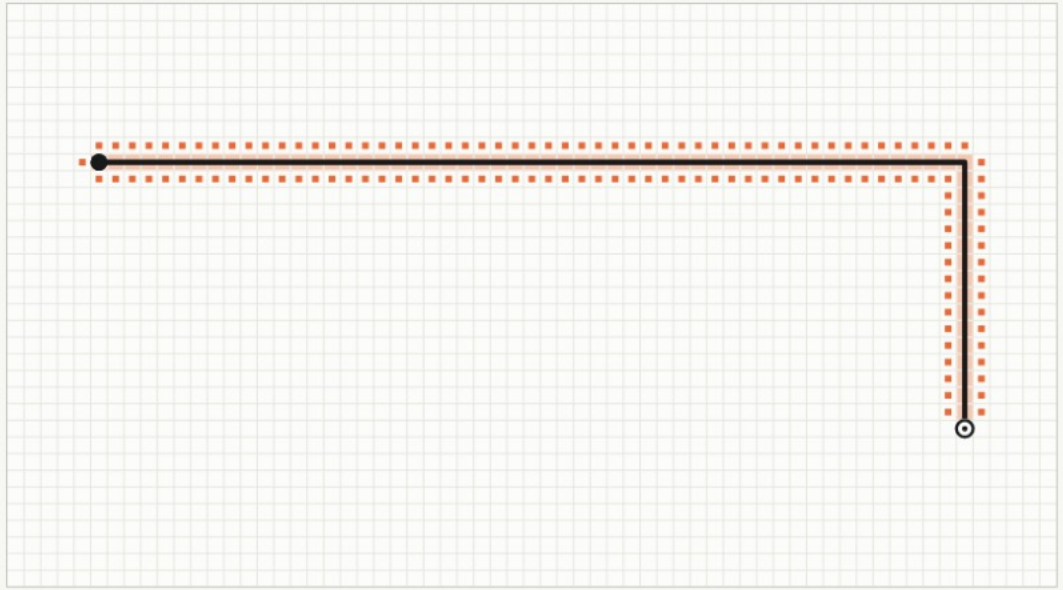
Open grid: Manhattan distance is exact

A* only settles cells that lie on a shortest path; the larger_g tie-break keeps it on one of them.

Dijkstra SETTLED 2,091



A* · Manhattan SETTLED 69



REPLAY PLAY

This run: A* settled **30× fewer** cells than Dijkstra. Both paths: **68** steps.

222×
fewer expanded vertices with A*

106×
faster in wall-clock time

100%
of queries where A* was faster

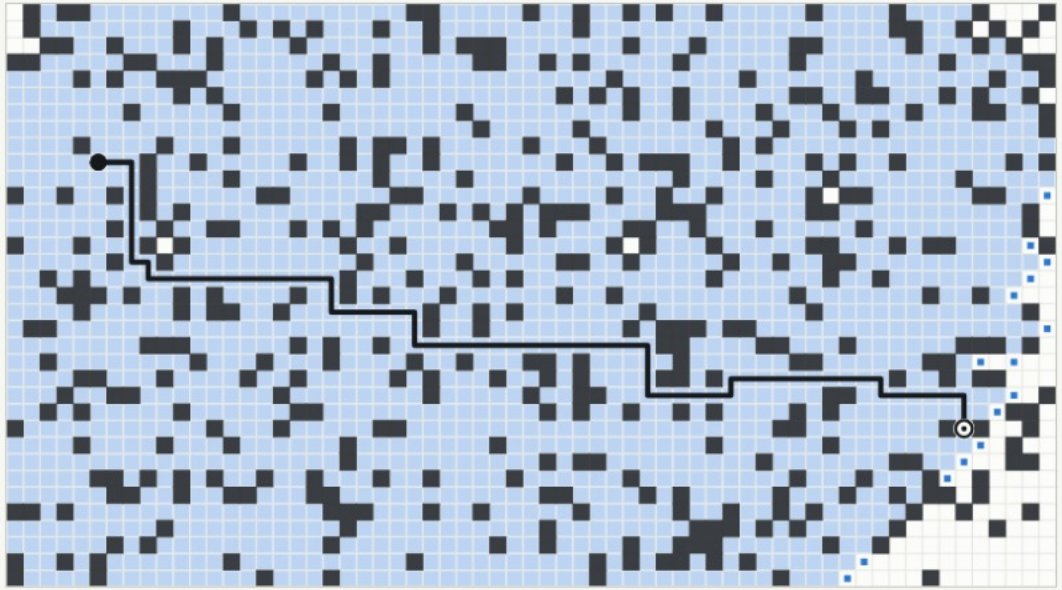
2.03×
cost of one A* expansion vs one Dijkstra expansion

numbers: 60 queries, 128² to 1024², medians

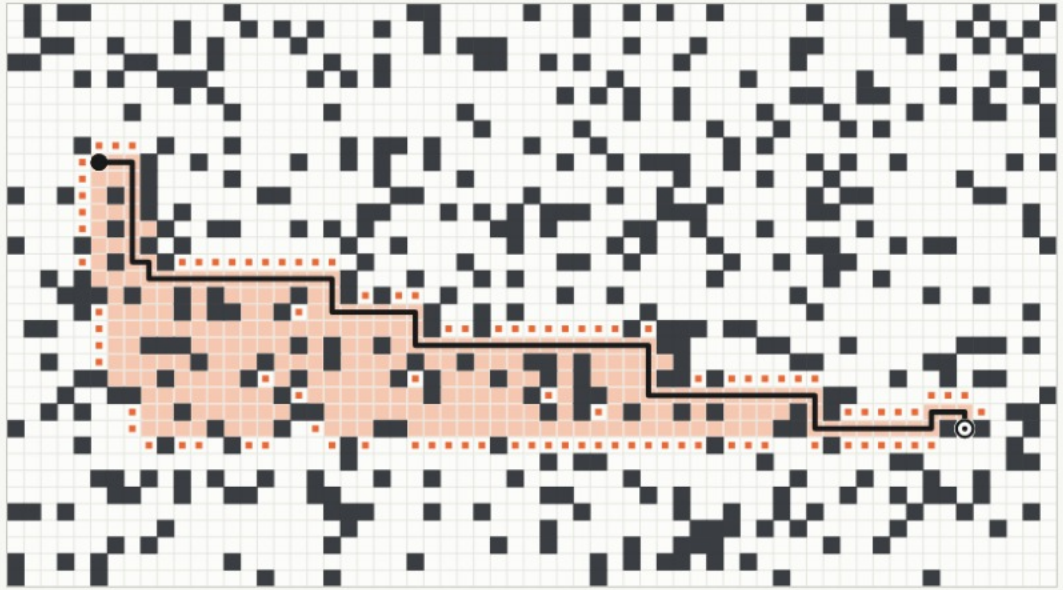
Random walls: the guess is slightly optimistic

Walls only make paths longer, so Manhattan stays a safe lower bound. Small detours make A* spread a little.

Dijkstra SETTLED 1,646



A* · Manhattan SETTLED 263



REPLAY PLAY This run: A* settled **6.3× fewer** cells than Dijkstra. Both paths: **70** steps.

21.4×
fewer expanded vertices with A*

12.8×
faster in wall-clock time

98%
of queries where A* was faster

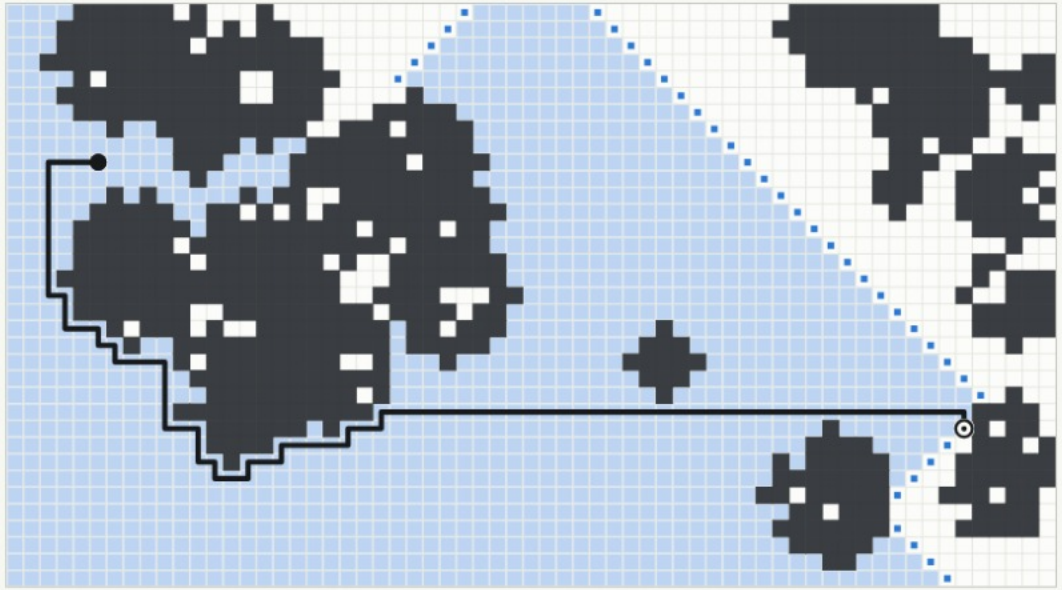
1.62×
cost of one A* expansion vs one Dijkstra expansion

numbers: 60 queries, 128² to 1024², medians

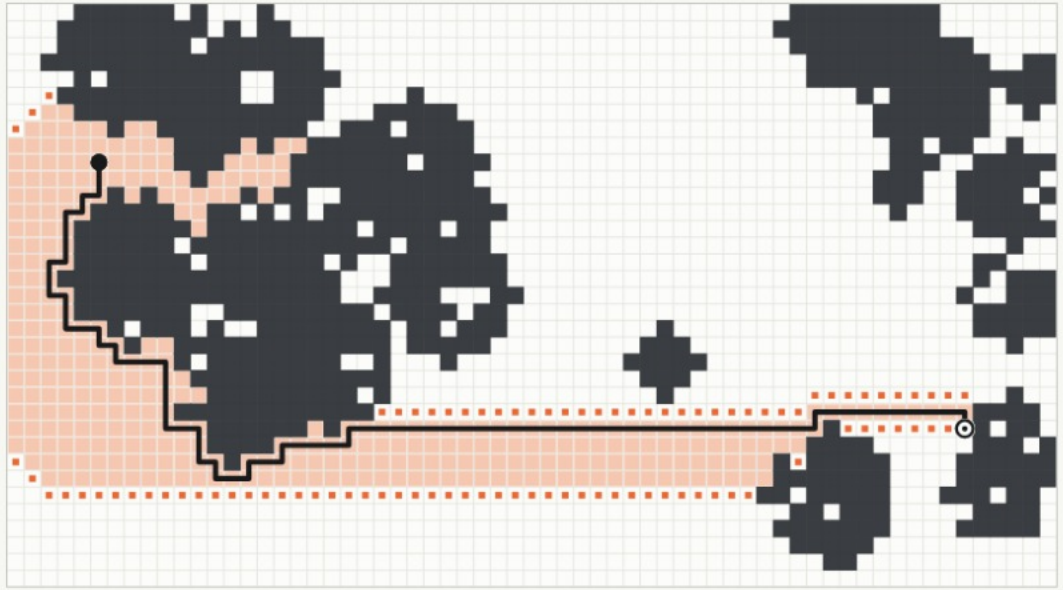
Clustered walls: A* bumps into blobs and goes around

Behind a blob the guess is too small, so A* fans out along its edge. Most of the grid is still never touched.

Dijkstra SETTLED 1,144



A* · Manhattan SETTLED 339



REPLAY PLAY This run: A* settled **3.4× fewer** cells than Dijkstra. Both paths: **82** steps.

22.9×
fewer expanded vertices with A*

12.8×
faster in wall-clock time

98%
of queries where A* was faster

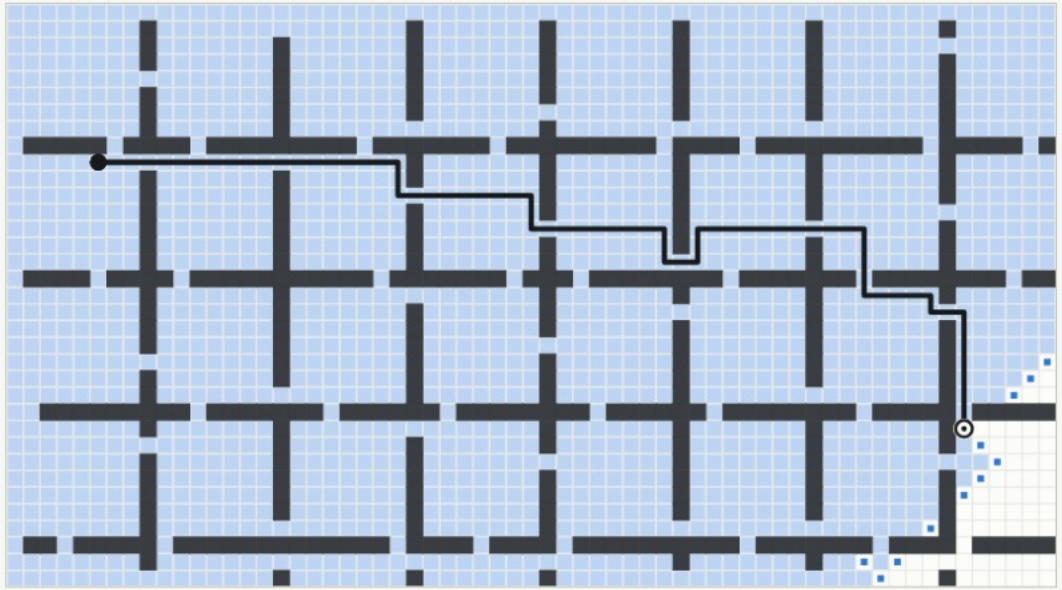
1.93×
cost of one A* expansion vs one Dijkstra expansion

numbers: 60 queries, 128² to 1024², medians

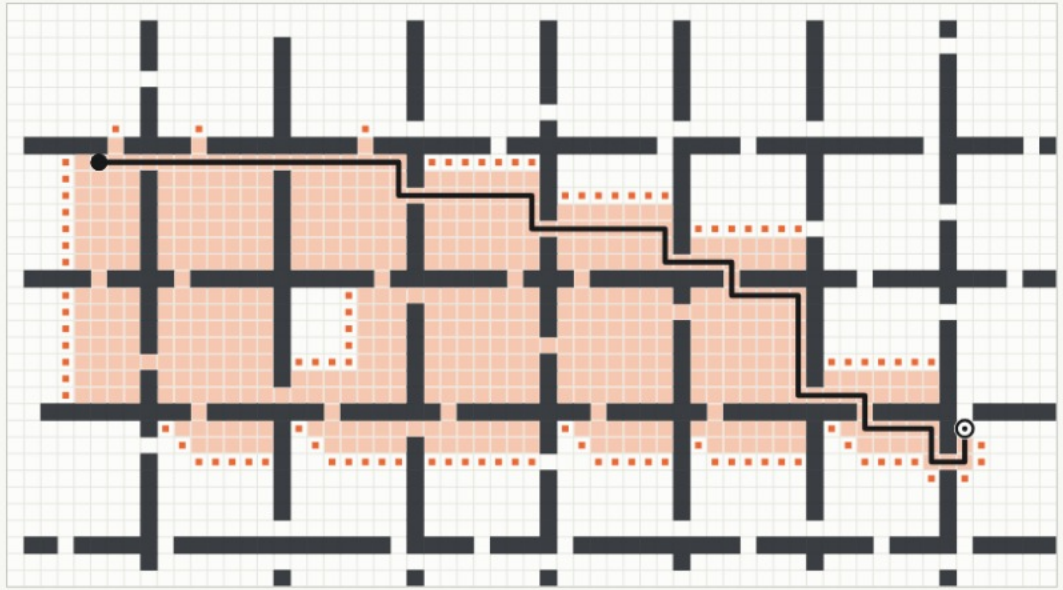
Rooms: every route has to pass through doors

The straight line runs into a wall, and A* fills part of each room until it finds the next door.

Dijkstra SETTLED 1,748



A* · Manhattan SETTLED 580



REPLAY PLAY

This run: A* settled **3.0× fewer** cells than Dijkstra. Both paths: **72** steps.

14.3×
fewer expanded vertices with A*

8.7×
faster in wall-clock time

100%
of queries where A* was faster

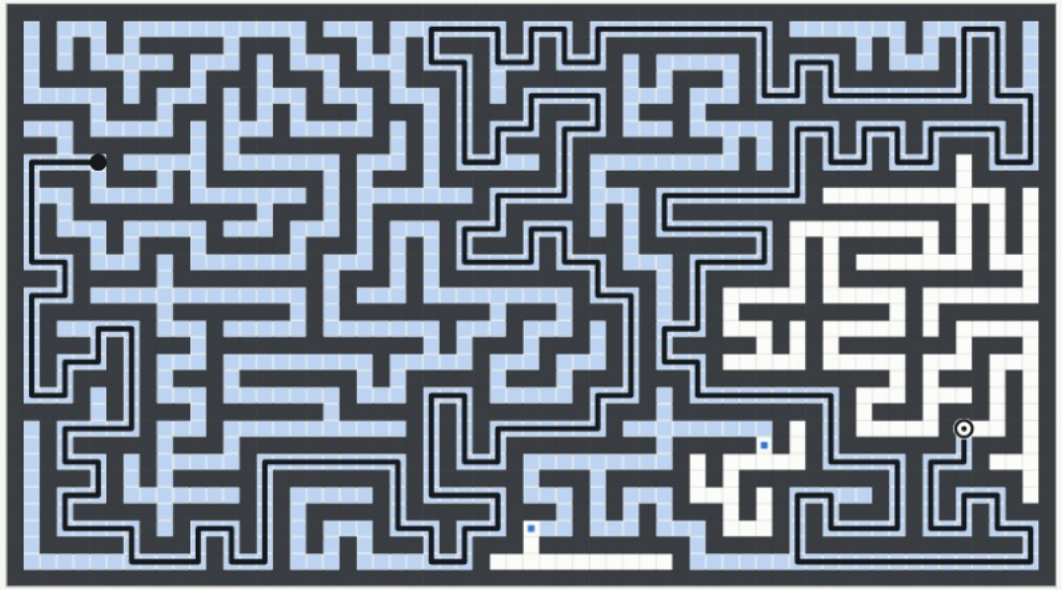
1.85×
cost of one A* expansion vs one Dijkstra expansion

numbers: 60 queries, 128² to 1024², medians

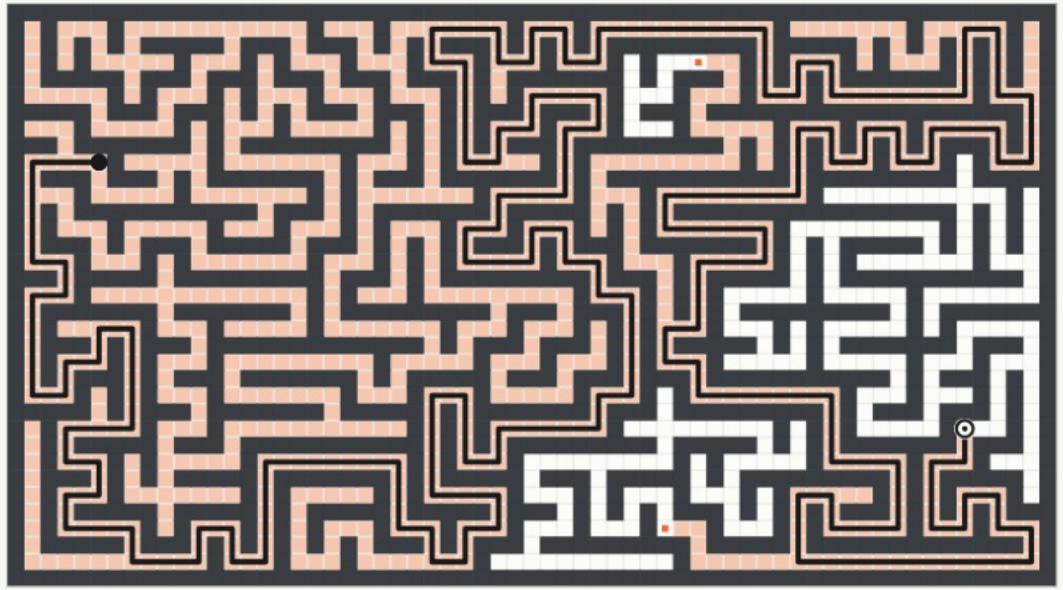
Maze: the guess points through walls

Manhattan sees a straight line, but the only route winds. A* settles almost the same cells and pays extra for each one.

Dijkstra SETTLED **884**



A* · Manhattan SETTLED **831**

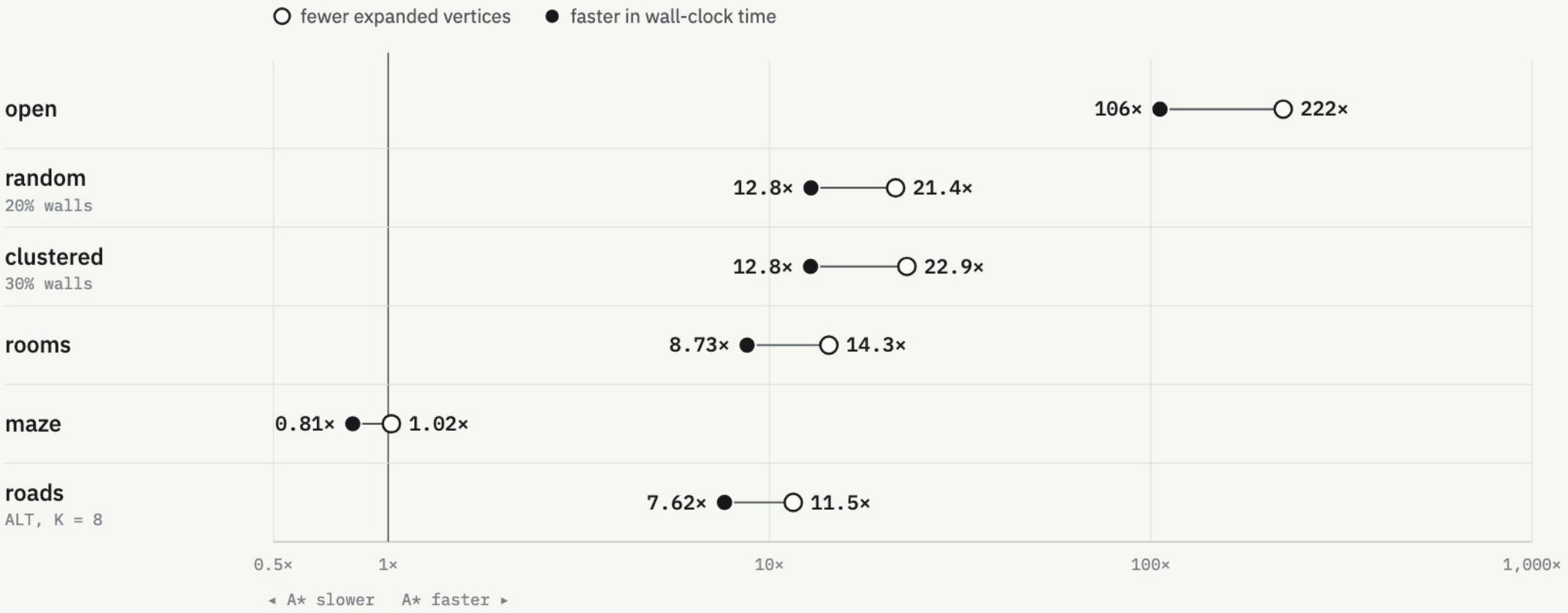


REPLAY PLAY Both settled about the same number of cells (884 vs 831). Both paths: **384** steps.

1.02×	**0.81×**	**17%**	**1.21×**
fewer expanded vertices: almost none	time ratio: A* is **24% slower**	of queries where A* was faster	cost of one A* expansion vs one Dijkstra expansion

ratio Dijkstra ÷ A*, log scale, medians

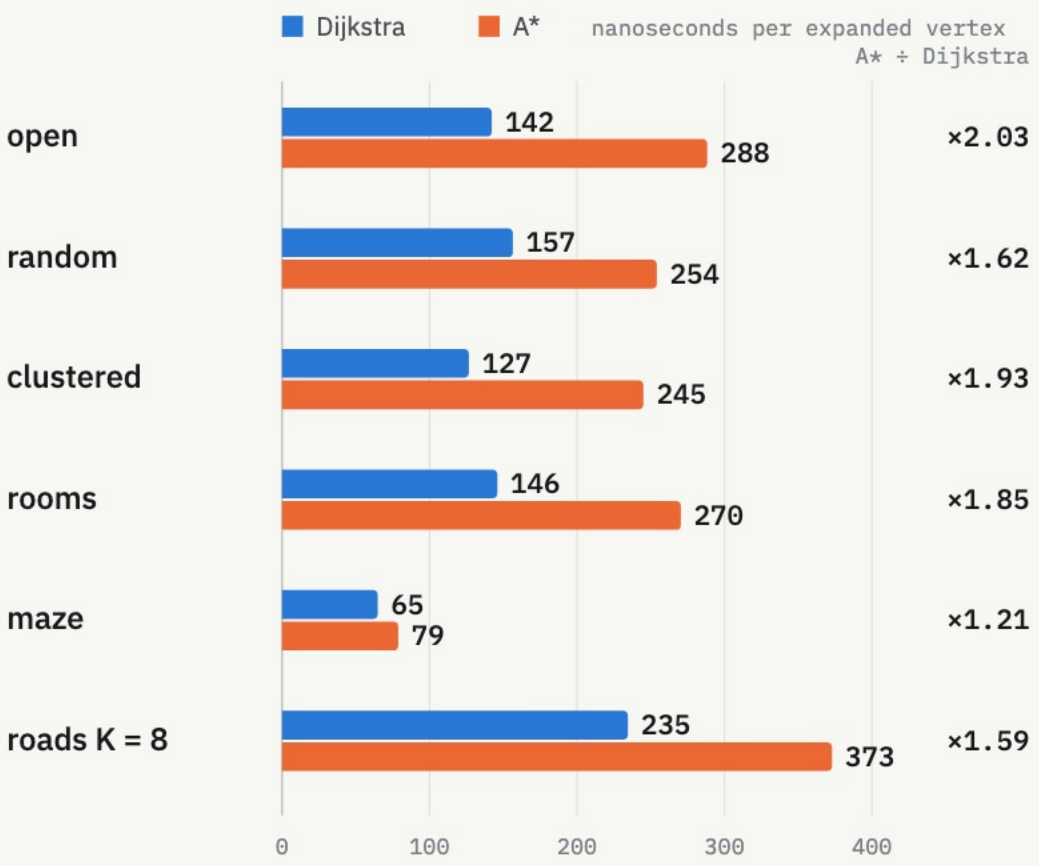
Structure decides: from 106× faster to 24% slower



In every row the dot (time) sits left of the ring (expansions): A* always gains less time than it saves in expansions. The next slide shows why.

medians per query · ns = nanoseconds

Each A* step costs more, so it must save more steps



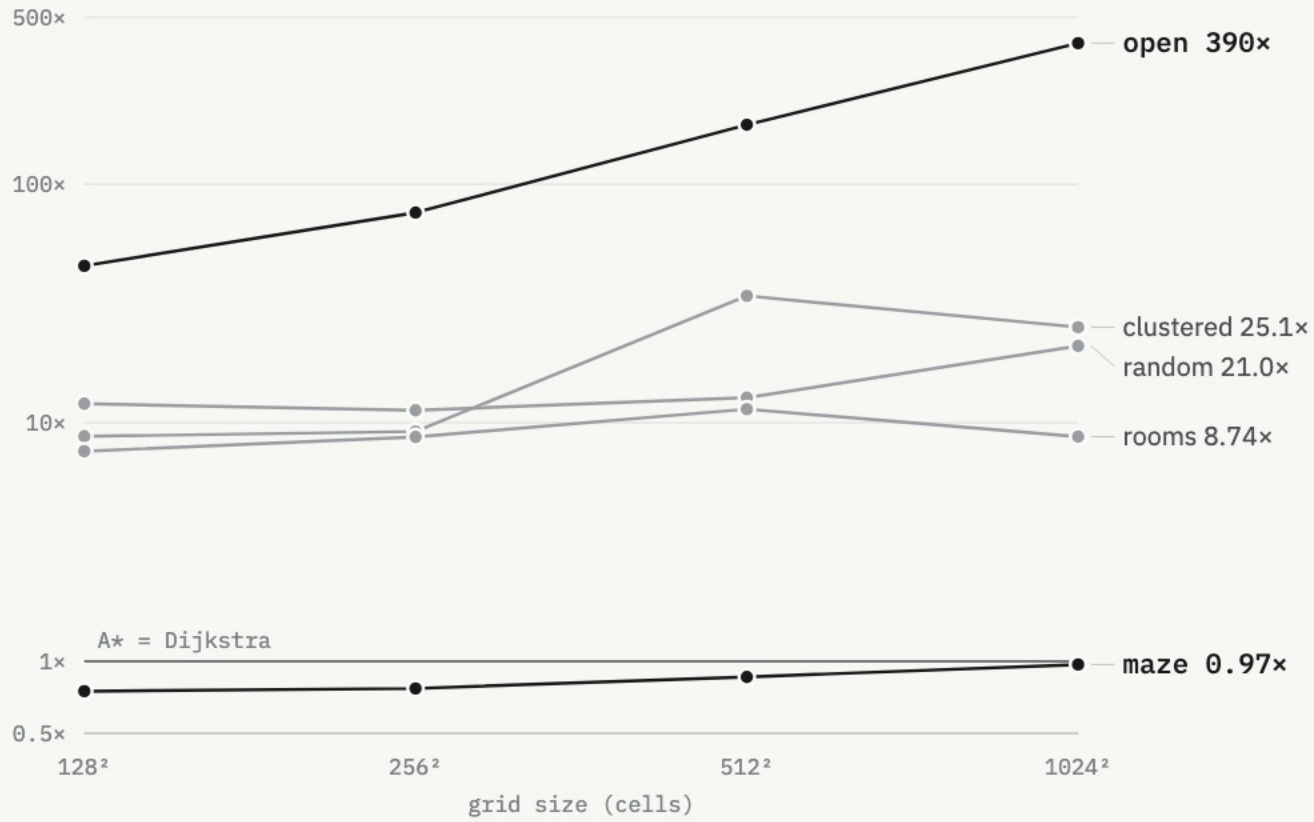
GRAPH	FEWER EXPANSIONS	COSTLIER STEP	PREDICTED = ÷	MEASURED TIME RATIO
open	222×	2.03×	109×	**106×**
random	21.4×	1.62×	13.2×	**12.8×**
clustered	22.9×	1.93×	11.9×	**12.8×**
rooms	14.3×	1.85×	7.7×	**8.7×**
maze	1.02×	1.21×	0.84×	**0.81×**
roads, ALT K = 8	11.5×	1.59×	7.3×	**7.6×**

Fewer expansions ÷ costlier step predicts the measured speed-up within 13% in every case. In the maze the first number is 1.02, so the extra cost per step decides the result.

Why a step costs more: for each vertex it settles, A* pushes 1.6 to 3.0 heap entries (Dijkstra: 1.0) and computes h for each one. These are neighbours at the edge of its narrow search that it never settles. Open grid: 3.0 pushes, 2.03× cost; maze: 1.0 push, 1.21×.

time ratio Dijkstra ÷ A*, 15 queries per point, log scale

The open-grid win grows with size; walls cap it



On the open grid Dijkstra's work grows with the area (about $2d^2$ cells for distance d) while A* grows with the path (about d cells), so the ratio grows like d: **45× at 128^2**, **390× at 1024^2**. Eight times the side gave 8.6 times the ratio.

With walls the ratio stays between 8× and 34× at every size: detours keep the A* search wider than one path. The maze approaches 1 but never crosses it in the median.

time ratio / expansion ratio, Dijkstra ÷ A*, medians

Longer queries mean more work, not always a bigger saving

GRAPH	SHORT 8–25% OF MAX	MEDIUM 40–60%	LONG 80–97%
open	83.5× / 150×	150× / 298×	**156× / 295×**
random	**14.4× / 22.2×**	12.8× / 20.1×	11.5× / 21.6×
clustered	11.9× / 20.9×	14.3× / 24.3×	12.0× / 22.8×
rooms	10.3× / 11.9×	6.9× / 10.6×	9.5× / 20.8×
maze	0.82× / 1.05×	0.75× / 1.01×	0.84× / 1.00×
roads, ALT K = 8	**14.9× / 19.1×**	9.4× / 13.5×	4.3× / 7.5×

Only the open grid gains clearly with distance, on both machines.

On random grids the long-query saving is slightly **lower** than the short one, and on roads it falls from 14.9× to 4.3×.

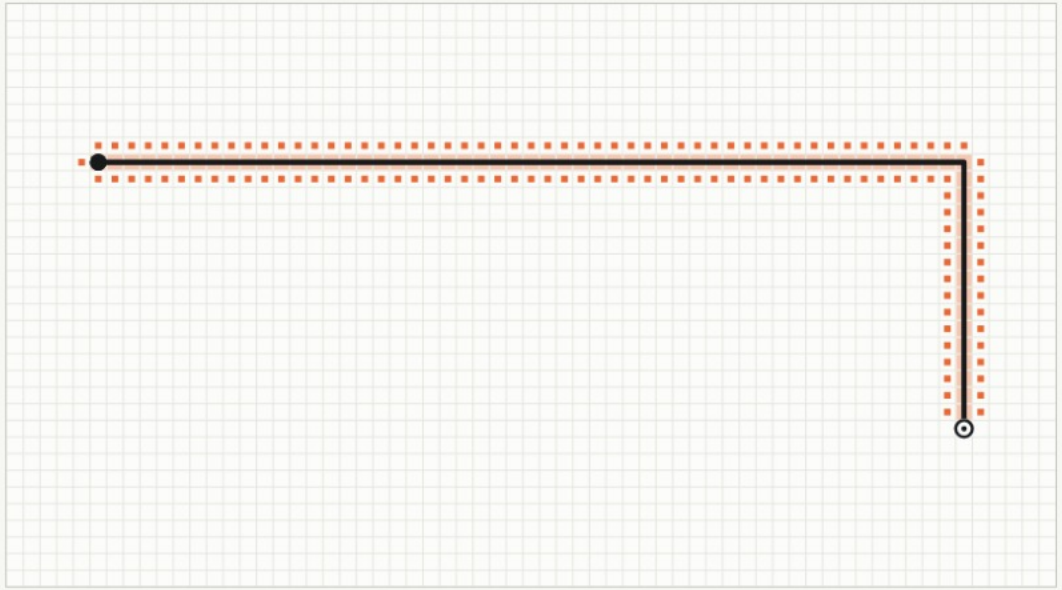
Neither trend came back in the replication (random long 26.8× vs short 20.9×; roads 10.0×, 9.8×, 13.2×). With 5 to 10 queries per cell, distance effects are noise next to structure.

numbers: 60 open-grid queries per machine, expansions node_id ÷ larger_g

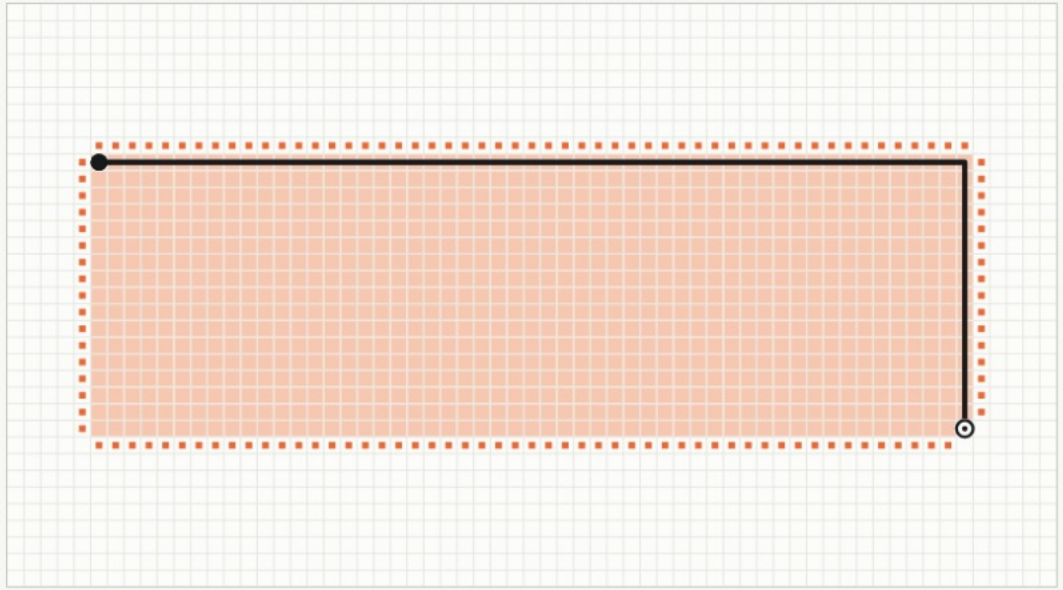
Tie-breaking: node_id fails whenever the target is below

Many cells tie on f. `larger_g` keeps walking; `node_id` takes the lowest cell number: the top rows.

A* · larger_g SETTLED **69**



A* · node_id SETTLED **901**



REPLAY | PLAY | TARGET BELOW START | TARGET ABOVE START

This run: larger_g settled **13× fewer** cells than node_id. Both paths: **68** steps.

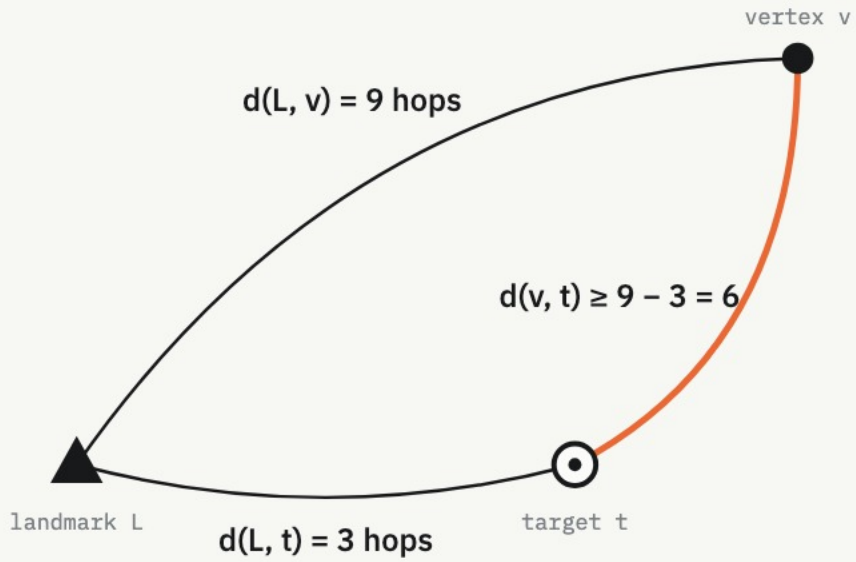
72 of 73
queries with the target below the start got worse with node_id (both machines)

0 of 47
queries with the target above or level got worse

286×
worst single query: node_id expanded this many times more

1.00× vs 35.8×
median on the two machines: 47% vs 75% of targets were below

Roads have no grid coordinates, so landmarks give the lower bound



Before any query: pick 16 landmarks far from each other and store the BFS distance from each landmark to every vertex (16 × 1.96 M numbers, 125 MB).

Triangle inequality: going from L to v through t is never shorter than the shortest route, `d(L,v) ≤ d(L,t) + d(t,v)`, so `d(v,t) ≥ d(L,v) − d(L,t)`.

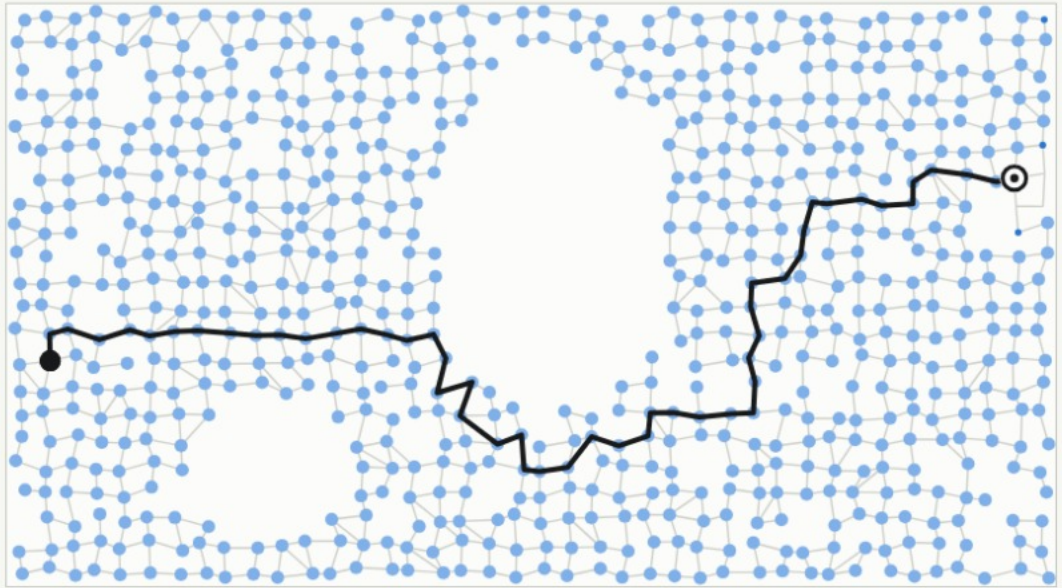
```
h(v) = max over landmarks |d(L,v) − d(L,t)|
```

It never overestimates and is consistent, so A* still returns the true shortest path. The experiment uses the first **K = 1, 4, 8 or 16** landmarks; every h(v) reads K table entries.

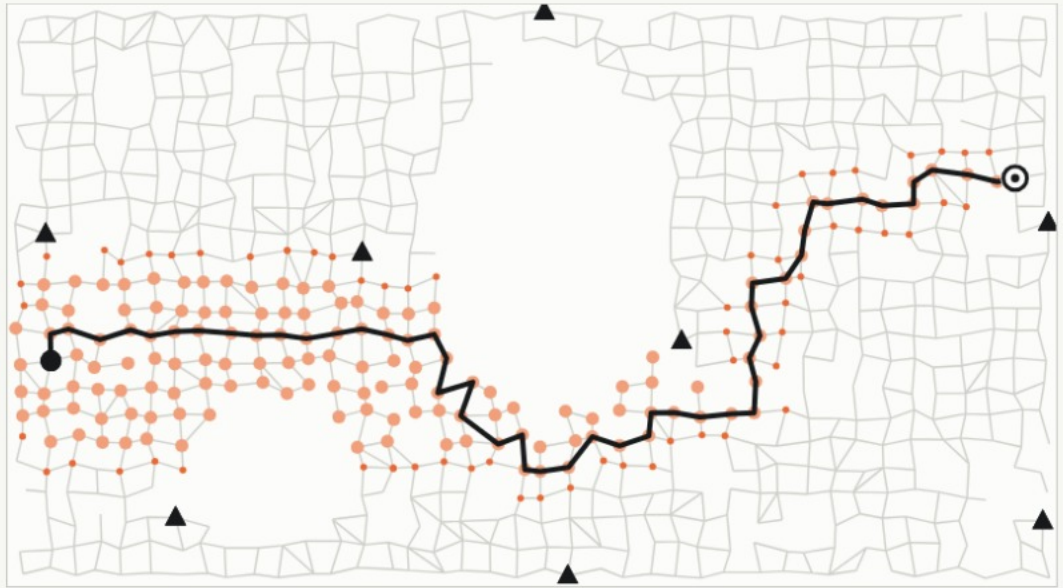
illustration: synthetic street grid with two lakes, unit hop cost

ALT pulls the search toward the target; more landmarks pull harder

Dijkstra SETTLED 716



A* · ALT, K = 8 SETTLED 146



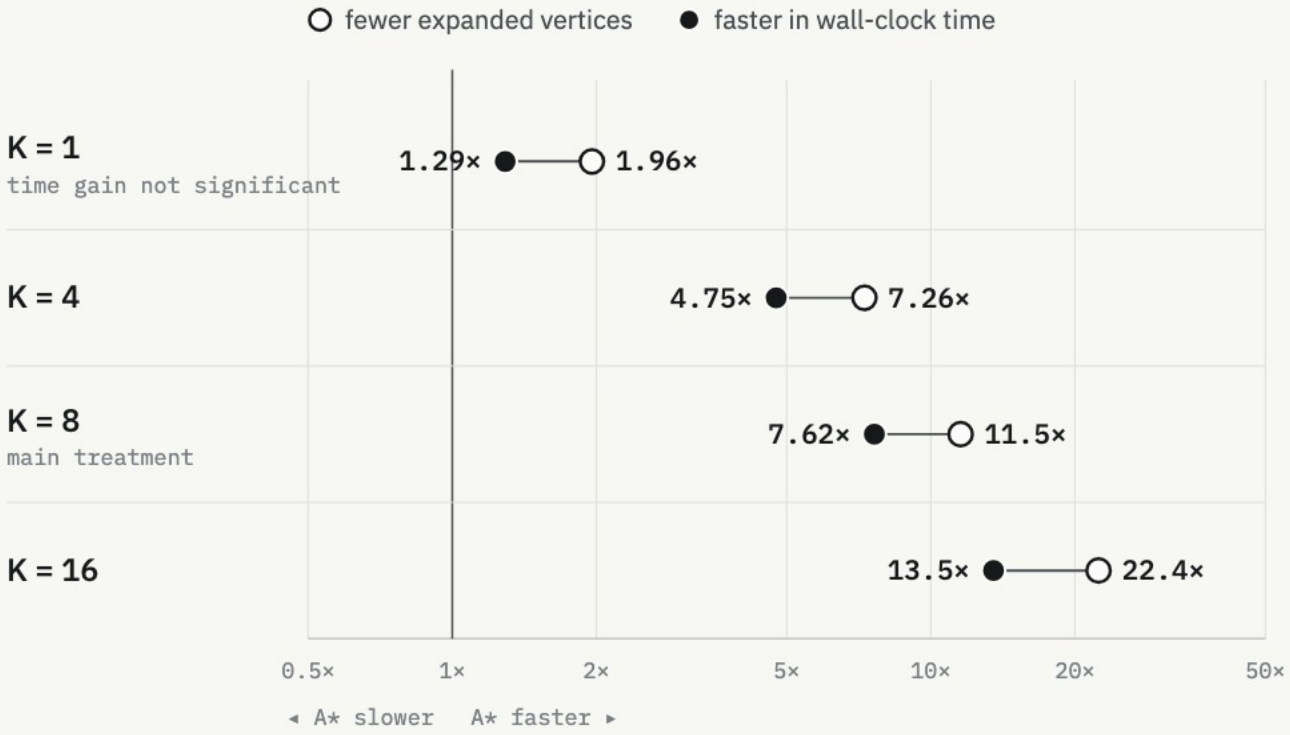
REPLAY | PLAY | K = 1 | K = 4 | K = 8 | K = 16

This run: A* settled **4.9× fewer** cells than Dijkstra. Both paths: **51** steps.

▲ landmark in use — shortest path the real graph has 1.96 M vertices; the numbers on the next slide come from it

roadNet-CA · 30 queries × 5 repeats · ratio Dijkstra ÷ A*, log scale

More landmarks, more savings, with no plateau at 16



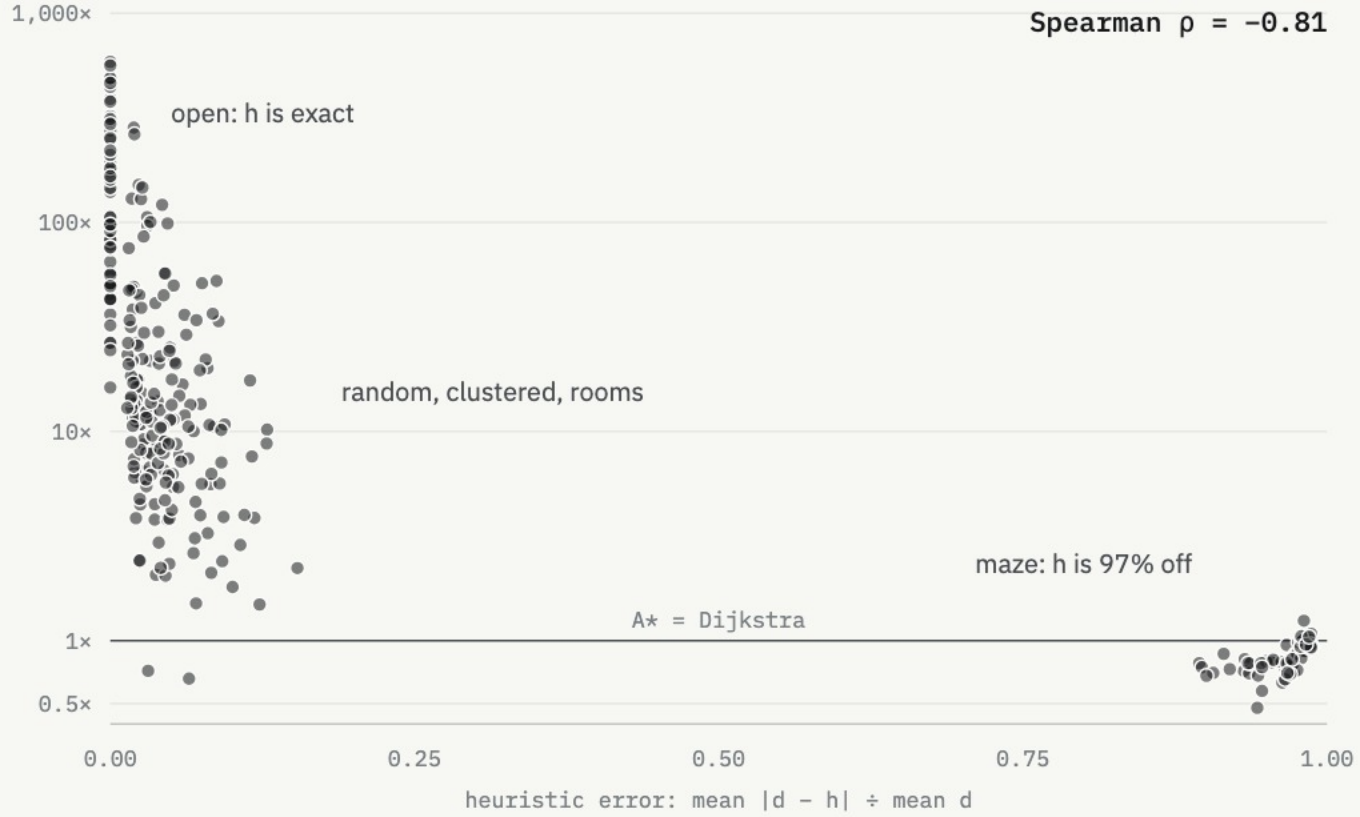
K = 1 halves the expansions, but the time gain is small: not significant here ($p = 0.13$), 1.41× with $p = 0.007$ on the second machine.

K = 16 vs K = 8 on the same queries: 21% less time and 25% fewer expansions ($p < 10^{-4}$). The predicted plateau did not appear.

Long queries (K = 8): Dijkstra 448 ms, A* 99 ms.

300 grid queries · error measured on 5,000 sampled vertices each

Admissible is not enough: the guess has to be close



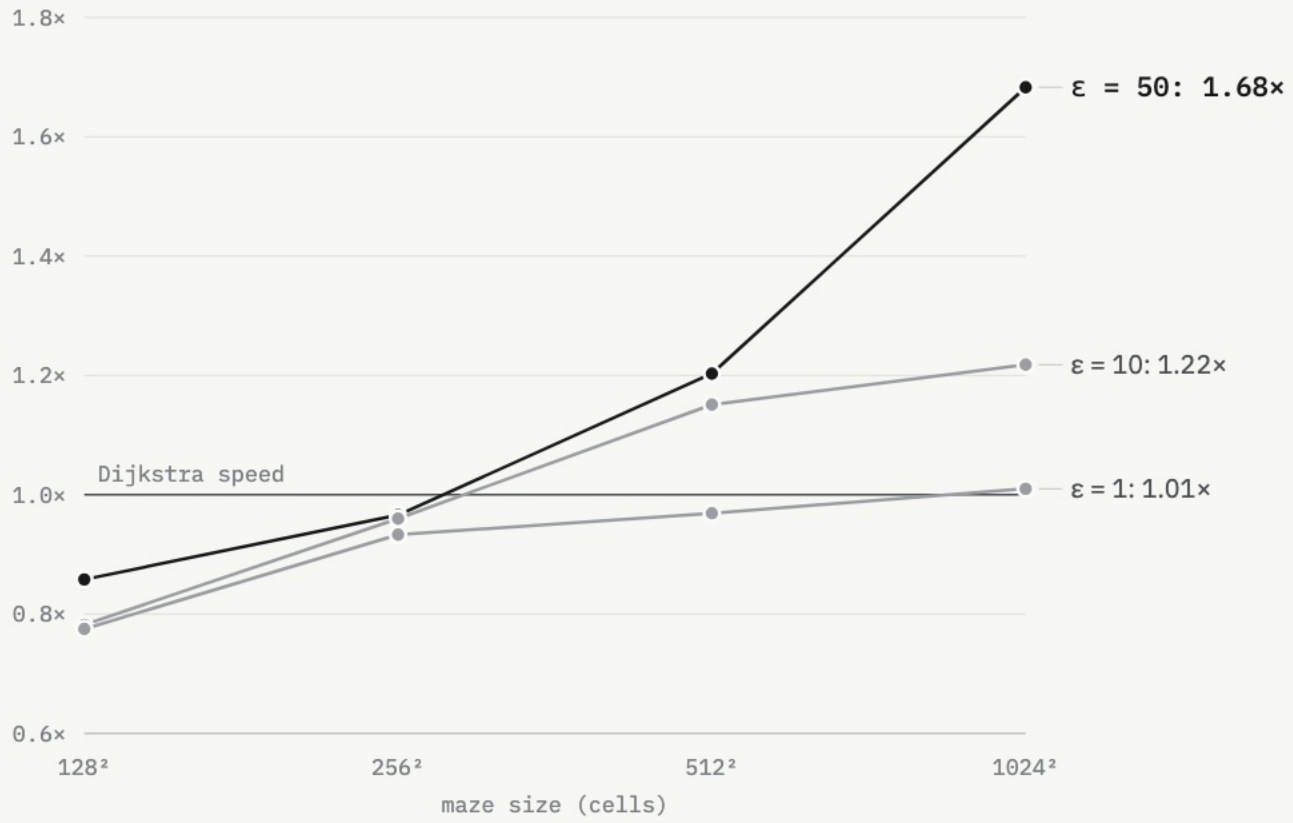
Heuristic error = mean |true distance − h| ÷ mean true distance, from a reverse BFS from the target.

Spearman correlation with the speed-up: **−0.81** (−0.83 on the second machine). A straight-line fit on this one number explains 55% of the spread in log speed-up.

Maze: error **0.97**, so h is almost useless. Roads with ALT K = 8: error **0.155**, rank correlation with the true distance **0.98**.

original run · f = g + ε·h · time ratio Dijkstra ÷ weighted A*, 15 queries per point

In large mazes a greedier A* wins, and the path stays optimal



Multiplying h by ε makes A* chase the target harder. It loses the guarantee of a shortest path.

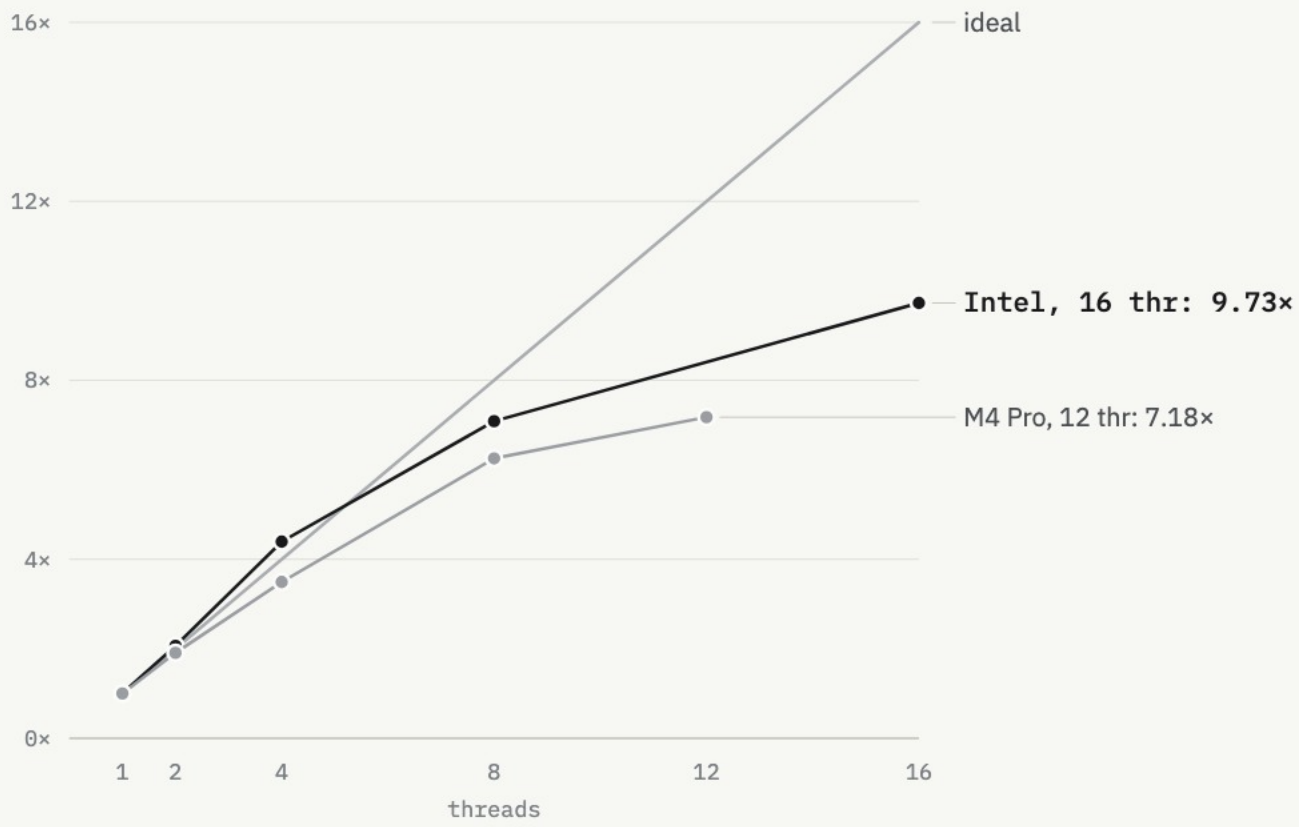
A perfect maze has exactly one route between two cells, so the path cost stayed **1.00 at every ε**.

At 1024², ε = 50 is **1.68× faster** than Dijkstra (short queries 3.43×). At 128² it was still slower (0.86×), but 1.14× faster on the second machine, so the small-maze result is not stable.

With alternative routes, ε would trade path length for speed.

5 grids at 1024² + 30 road queries · 270 tasks per batch

Parallel batches: throughput scales, the A* advantage holds



Each search stays sequential. OpenMP hands whole queries to threads, each with its own scratch memory and heuristic.

16 threads: 9.73× throughput (61% efficiency) on the original machine; **12 threads: 7.18×** (60%) on the M4 Pro.

The median Dijkstra ÷ A* time ratio stays put under load: 14.8× → 17.9× and 14.6× → 15.4×. The maze stays below 1.

same code, same seeds, full pipeline rerun

A second machine confirms the main effects, not the small ones

GRAPH	TIME RATIO ORIGINAL	TIME RATIO REPLICATION	EXPANSIONS ORIGINAL	EXPANSIONS REPLICATION
open	106×	**209×**	222×	257×
random	12.8×	**15.6×**	21.4×	22.2×
clustered	12.8×	**12.5×**	22.9×	20.8×
rooms	8.7×	**8.6×**	14.3×	12.9×
maze	0.81×	**0.79×**	1.02×	1.01×
roads, ALT K = 1	1.29×	**1.41×**	1.96×	2.28×
roads, ALT K = 4	4.75×	**7.93×**	7.27×	11.2×
roads, ALT K = 8	7.62×	**10.6×**	11.5×	14.6×
roads, ALT K = 16	13.5×	**15.0×**	22.4×	24.1×

Original: Intel Core Ultra 9 285H, Ubuntu on WSL2, GCC 13. **Replication:** Apple M4 Pro, macOS, Apple Clang 21.

Same seeds, but the two standard libraries draw different random grids and queries, so this is a new sample. The road graph and landmark files were byte-identical (same SHA-256).

Held: the order of the five families, the maze slowdown, more landmarks always better, error vs speed-up ($\rho = -0.83$), 3,600 of 3,600 runs correct.

Moved: one A* step costs only 1.3–1.6× on the M4 Pro, so open-grid A* gained 209×.

Did not hold: the distance-band trends, weighted A* on the smallest mazes, the tie-break median.

What held on both machines, and what did not

#	PREDICTION	ORIGINAL RUN	SECOND MACHINE	VERDICT
H1	Open grid: large saving in expansions, smaller in time	222× fewer, 106× faster, 100% of queries	257× fewer, 209× faster, 100%	✓ **Confirmed**
H2	Random walls: clear win, smaller than open	21.4× fewer, 12.8× faster, 98%	22.2× fewer, 15.6× faster, 100%	✓ **Confirmed**
H3	Rooms and clusters save less than open; rooms less than random	Pooled 14.3× vs 21.4×; fails at 512^2	Pooled 12.9× vs 22.2×; fails at 1024^2	✓ **Confirmed, pooled**
H4	Maze: equal expansions, A* slower	1.02× fewer, 0.81× time	1.01× fewer, 0.79× time	✓ **Confirmed**
H5	Long queries save more than short on open and random	Random long 21.6× < short 22.2×	Random long 26.8× > short 20.9×	◐ **Not robust**
H6	ALT: K = 8 wins, K = 1 weak, K = 16 plateaus	7.6×; K = 1 n.s.; K = 16 beats K = 8	10.6×; K = 1 1.41×; K = 16 beats K = 8	◐ **Partly: no plateau**
H7	larger_g lowers the median expansions on open grids	Median 1.00, tail 286×	Median 35.8×: more targets below	◐ **Median not robust**

Where the conclusions stop

One real graph, hop counts	SNAP gives no road lengths or travel times. Weighted routing could behave differently.
Synthetic grids	Each family isolates one structure. They are controls, not a sample of real maps.
Two-way roads	The road graph was symmetrised; one-way streets are not modelled.
One implementation	Binary heap with lazy deletion for both. Another priority queue changes the constant per step.
Few queries per cell	5 to 10 queries per size and distance band. Effects at that level did not replicate; only family-level results are claimed.
Wall-clock time	Absolute times depend on the machine; paired ratios, medians, bootstrap intervals and a second machine are the defence.

A* saves time when its heuristic removes more expansions than it adds cost to each one.

SITUATION	USE	EVIDENCE
Open or lightly obstructed grid	**A*, Manhattan**	12.8× to 106× faster
Rooms, clustered obstacles	**A*, Manhattan**	8.7× to 12.8× faster
Road network with 8 or more landmarks	**A*, ALT**	7.6× (K = 8) to 13.5× (K = 16) faster
Perfect maze, long winding corridors	**Dijkstra**	A* is 24% slower; weighted A* helps only on large mazes